

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf_00bebd3c-82f\agent-aea6e2227b701f5ec.jsonl`
- SHA-256: `e71916aa09c2018adea876529541fc259737a2e496d2a494ee2099af48bf8131`
- Source modified: `2026-07-08T05:52:33+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`
```

Transcript

1. user (2026-07-08T05:44:49.632Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).

Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.

Subsystem: UPLOAD FLOW. Map backend/api/uploads.py end to end: chunked upload endpoints, how MD5/video_id is computed incrementally during streaming, what upload_complete returns, where the bytes land (which StorageBackend call, gs:// layout), and the #480/#471 OOM-fix design (never touching CP local disk). Then map how the SPA/client submits /api/process afterwards: find the process endpoint in backend/main.py (or backend/api/), its request model fields (does it accept a video_id or just a URI/path?), and what it passes to orchestration. Also check the vendored SPA bundle or API client if present for what the frontend actually sends to /api/process after upload_complete.

2. user (2026-07-08T05:45:01.410Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2). Where they assemble depends
5      on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7      * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
8      file under security.upload_dir, exactly as before.
9      * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10     through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11     while an incremental MD5 accumulates ? the assembled file never touches local
12     disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
13     (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
14     returns the staged URI (inside the #465 input jail by construction ? it is under
15     ``input_root``) plus the streamed md5 as ``video_id``.
16
17     Upload state is in-process by design (this is the single-box alpha): a server restart
18     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
19     arrives with PR-3 identity scoping.
20
21     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
22     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
23     reads the live value the server set at startup (and so tests that monkeypatch
24     ``backend.main.global_config`` flow through unchanged).
```

```

25     """
26
27     import hashlib
28     import os
29     import threading
30     import unicodedata
31     import uuid
32     from typing import Any, Dict, Optional
33
34     from fastapi import APIRouter, HTTPException, Request
35     from pydantic import BaseModel
36     from starlette.concurrency import run_in_threadpool
37
38     from backend import storage
39     from backend.utils.app_home import (
40         resolve_output_dir, # P0a: app-home-aware path resolution
41     )
42     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
43     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
44     from backend.utils.paths import safe_output_filename
45
46     router = APIRouter()
47
48     _uploads: Dict[str, Dict[str, Any]] = {}
49     _uploads_lock = threading.Lock()
50
51
52     class UploadInitRequest(BaseModel):
53         filename: str
54         total_size_bytes: Optional[int] = None
55
56
57     class UploadCompleteRequest(BaseModel):
58         total_parts: int
59
60
61     def _upload_cfg():
62         # Resolved lazily (import inside the function) to read the LIVE startup config that
63         # main() sets on the module global, and to avoid a circular import at module load
64         # (main.py imports this router; this module must not import main at import time).
65         import backend.main as _main
66
67         sec = getattr(_main.global_config, "security", None)
68         upload_dir = resolve_output_dir(
69             getattr(sec, "upload_dir", None) or "output/uploads"
70         ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
71         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
72         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
73         exts = [
74             e.lower().lstrip(".")
75             for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
76         ]
77         return upload_dir, max_bytes, part_bytes, exts
78
79
80     def upload_caps(cfg) -> Dict[str, int]:
81         """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).
82
83         ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
84         ?
85         without it the guard is dead code and the user learns from a mid-upload 413.
86         ``max_part_mb`` is informational: each /api/upload/init response carries the
87         authoritative value the client actually chunks by."""
88         sec = getattr(cfg, "security", None)
89         return {

```

```

89         "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
90         "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
91     }
92
93
94 def _staging_target() -> "Optional[Dict[str, Any]]":
95     """The remote staging root + storage settings, or None for the local-file flow.
96
97     Remote staging is selected by the same input config /api/process resolves against
98     (storage.input_backend + input_root), so a staged upload is submittable verbatim.
99     """
100     import backend.main as _main
101
102     sc = getattr(_main.global_config, "storage", None)
103     if sc is None:
104         return None
105     input_backend = getattr(sc, "input_backend", "local") or "local"
106     input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107     if input_backend == "local" or not input_root:
108         return None
109     cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
110     return {"root": input_root, "config": cfg}
111
112
113 def _abort_upload(upload_id: str) -> None:
114     with _uploads_lock:
115         st = _uploads.pop(upload_id, None)
116         if not st:
117             return
118         writer = st.get("writer")
119         if writer is not None:
120             # A streaming writer has no cancel ? close() finalizes whatever was staged, then
121             # the partial object is deleted. Both are best-effort: the upload is already
122             # aborted from the client's point of view.
123             try:
124                 writer.close()
125             except Exception: # noqa: BLE001 ? abort cleanup must never mask the abort
126                 pass
127             try:
128                 ref = storage.parse_uri(st["stage_uri"])
129                 storage.get_storage(ref.backend, config=st.get("config")).delete(ref)
130             except Exception: # noqa: BLE001
131                 pass
132             return
133         try:
134             os.remove(st["tmp_path"])
135         except OSError:
136             pass
137
138
139 def _normalize_upload_filename(filename: str) -> str:
140     """Normalize browser-provided file names before extension validation.
141
142     Some mobile/file-provider pickers can hand us visually normal names with
143     compatibility
144     characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
145     existing
146     traversal/null-byte checks in ``safe_output_filename`` as the authority after this
147     cleanup.
148     """
149     return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
150
151 @router.post("/api/upload/init")
152 def upload_init(req: UploadInitRequest):

```

```

151 """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
152 upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
153 try:
154     name = safe_output_filename(_normalize_upload_filename(req.filename))
155 except ValueError as e:
156     raise HTTPException(status_code=400, detail=str(e)) from e
157 ext = os.path.splitext(name)[1].lower().lstrip(".")
158 if ext not in exts:
159     raise HTTPException(
160         status_code=400,
161         detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
162     )
163 if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
164     raise HTTPException(
165         status_code=413,
166         detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
167     )
168 upload_id = uuid.uuid4().hex
169 staging = _staging_target()
170 if staging is not None:
171     # Remote staging: stream parts straight into the bucket under the input root ?
172     # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
173     # upload_id in the key keeps concurrent same-name uploads distinct, and the
174     # space guard is skipped: no local bytes are consumed.
175     stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
176     try:
177         writer = storage.open_writer(stage_uri, config=staging["config"])
178     except NotImplementedError:
179         staging = None # backend can't stream (e.g. mounted-Drive) ? stage locally
180     else:
181         with _uploads_lock:
182             _uploads[upload_id] = {
183                 "filename": name,
184                 "writer": writer,
185                 "hasher": hashlib.md5(),
186                 "stage_uri": stage_uri,
187                 "config": staging["config"],
188                 "parts": 0,
189                 "bytes": 0,
190             }
191         return {
192             "upload_id": upload_id,
193             "max_part_mb": part_bytes // (1024 * 1024),
194             "next_part": f"/api/upload/{upload_id}/part/0",
195         }
196 os.makedirs(upload_dir, exist_ok=True)
197 # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
198 # effort ? an
199 # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
200 # (never blocks a
201 # legit upload); the per-part total cap in upload_part is the hard backstop either
202 # way.
203 require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
204 tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
205 open(tmp_path, "wb").close()
206 with _uploads_lock:
207     _uploads[upload_id] = {
208         "filename": name,
209         "tmp_path": tmp_path,
210         "parts": 0,
211         "bytes": 0,
212     }
213 return {
214     "upload_id": upload_id,

```

```

212         "max_part_mb": part_bytes // (1024 * 1024),
213         "next_part": f"/api/upload/{upload_id}/part/0",
214     }
215
216
217 @router.put("/api/upload/{upload_id}/part/{index}")
218 async def upload_part(upload_id: str, index: int, request: Request):
219     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
220     _, max_bytes, part_bytes, _ = _upload_cfg()
221     with _uploads_lock:
222         st = _uploads.get(upload_id)
223         if st is None:
224             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
225         if index != st["parts"]:
226             raise HTTPException(
227                 status_code=409,
228                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
229             )
230
231         received = 0
232         overflow = None
233         if st.get("writer") is not None:
234             # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
an
235             # oversize stream), then hand it to the blocking bucket writer in one threadpool
236             # hop so a network flush never stalls the event loop mid-upload.
237             buf = bytearray()
238             async for chunk in request.stream():
239                 received += len(chunk)
240                 if received > part_bytes:
241                     overflow = (
242                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
243                     )
244                     break
245                 if st["bytes"] + received > max_bytes:
246                     overflow = (
247                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
248                     )
249                     break
250                 buf += chunk
251             if not overflow and buf:
252                 data = bytes(buf)
253                 await run_in_threadpool(st["writer"].write, data)
254                 st["hasher"].update(data)
255         else:
256             with open(st["tmp_path"], "ab") as fh:
257                 async for chunk in request.stream():
258                     received += len(chunk)
259                     if received > part_bytes:
260                         overflow = (
261                             f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
262                         )
263                         break
264                     if st["bytes"] + received > max_bytes:
265                         overflow = (
266                             f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
267                         )
268                         break
269                     fh.write(chunk)
270         if overflow:
271             _abort_upload(upload_id)
272             raise HTTPException(

```

```

273         status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
274     )
275
276     with _uploads_lock:
277         st["parts"] += 1
278         st["bytes"] += received
279     return {
280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
289         /api/process."""
290         upload_dir, _, _, _ = _upload_cfg()
291         with _uploads_lock:
292             st = _uploads.get(upload_id)
293             if st is None:
294                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
295             if st["parts"] == 0 or st["parts"] != req.total_parts:
296                 raise HTTPException(
297                     status_code=409,
298                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
299                 )
300             if st.get("writer") is not None:
301                 # Remote staging: close() finalizes the streamed object; the md5 accumulated per
302                 # part IS compute_video_id's digest (same bytes, same algorithm), so
303                 # gets the identity without anyone re-reading the video. (Sync endpoint ?
304                 # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
305                 try:
306                     st["writer"].close()
307                 except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
308                     _abort_upload(upload_id)
309                     raise HTTPException(
310                         status_code=502,
311                         detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
312                     ) from e
313             with _uploads_lock:
314                 _uploads.pop(upload_id, None)
315             return {
316                 "path": st["stage_uri"],
317                 "video_id": st["hasher"].hexdigest(),
318                 "size_bytes": st["bytes"],
319                 "next": "POST /api/process with this path",
320             }
321         final_path = os.path.join(upload_dir, st["filename"])
322         if os.path.exists(
323             final_path
324         ): # never overwrite an earlier upload with the same name
325             stem, ext = os.path.splitext(st["filename"])
326             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
327         os.replace(st["tmp_path"], final_path)
328         with _uploads_lock:
329             _uploads.pop(upload_id, None)
330         video_id = compute_video_id(final_path)
331         return {
332             "path": os.path.abspath(final_path),
333             "video_id": video_id,
334             "size_bytes": os.path.getsize(final_path),
335             "next": "POST /api/process with this path",

```

```

335         }
336
337
338     @router.delete("/api/upload/{upload_id}")
339     def upload_abort(upload_id: str):
340         """Abort an in-flight upload and delete its partial file."""
341         with _uploads_lock:
342             known = upload_id in _uploads
343             if not known:
344                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345             _abort_upload(upload_id)
346             return {"status": "aborted", "upload_id": upload_id}
347

```

3. user (2026-07-08T05:45:04.634Z)

```

backend\api\__init__.py
backend\api\auth.py
backend\api\jobs.py
backend\api\uploads.py
backend\api\job_worker.py

```

4. user (2026-07-08T05:45:14.230Z)

```

Found 10 files
backend\orchestration.py
backend\infrastructure\database.py
backend\config\models.py
backend\main.py
backend\infrastructure\database_jobs.py
backend\api\job_worker.py
backend\api\uploads.py
backend\api\jobs.py
backend\config\startup.py
backend\api\static\app.js

```

5. user (2026-07-08T05:45:15.185Z)

```

backend\storage\base.py:66:     def open_writer(
backend\storage\gcs.py:95:     def open_writer(self, ref, *, content_type=None):
backend\storage\__init__.py:36:     "open_writer",
backend\storage\__init__.py:233: def open_writer(
backend\storage\__init__.py:241:     ``StorageBackend.open_writer``)."""
backend\storage\__init__.py:243:     return get_storage(ref.backend, config=config).open_writer(

```

6. user (2026-07-08T05:45:21.884Z)

```

94: from backend.utils.hashing import compute_video_id # canonical file-identity hashing
258: class ProcessRequest(BaseModel):
277:     # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
verbatim.
278:     video_id: str
289:     # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
297:     video_id: str
305:     video_id: str
439:     "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
448:         "video_id": row.get("id"),
461:     (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
482:     does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
512:     """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).

```

```

549: def _source_cache_lock(video_id: str) -> threading.Lock:
551:     return _source_cache_locks.setdefault(video_id, threading.Lock())
554: def _source_cache_path(video_id: str, input_ref, cfg) -> str:
559:     name = safe_output_filename(f"{video_id}{ext}")
563: def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:     cache_path = _source_cache_path(video_id, input_ref, cfg)
568:     lock = _source_cache_lock(video_id)
591: def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
596:     if video_id in _proxy_inflight:
598:         _proxy_inflight.add(video_id)
605:         src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608:         logger.warning("proxy: warm failed for %s: %s", video_id, e)
611:         _proxy_inflight.discard(video_id)
614: @app.get("/api/source/{video_id}")
615: def get_source(video_id: str, request: Request):
621:     rec = request.app.state.db.get_video(video_id)
628:     proxy_name = safe_output_filename(f"{video_id}.mp4")
655:     local = _cached_remote_source(video_id, input_ref, cfg)
667:     target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
677:     ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?
so it joins
681:     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
691:     name = safe_output_filename(f"{req.video_id}.rallies.csv")
693:     raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
707:     "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
720: def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721:     """"The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``. """"
722:     return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
748:     Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local
file from
753:     MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
822:     video_id = compute_video_id(local_working)
844:     dest = _asset_dest_uri(req.medium_uri, video_id, ext)
865:     "[ASSETS] put failed video_id=%s dest=%s: %s",
866:     video_id,
876:     if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
879:     "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
880:     video_id,
889:     "video_id": video_id,
904: @app.post("/api/process", status_code=202)
905: def process_video(req: ProcessRequest, request: Request):
992: @app.get("/api/videos/{video_id}/cost")
993: def get_video_cost(video_id: str, request: Request):
996:     agg = request.app.state.db.get_video_cost(video_id)
1015:     "video_id": job.get("video_id"),
1041: @app.get("/api/highlights/{video_id}/{filename}")
1042: def download_highlight(video_id: str, filename: str, request: Request):
1050:     if not request.app.state.db.get_video(video_id):
1057:         signed = signed_reel_url(request.app.state.config, video_id, name)
1061:         logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1090:     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1092:     request.app.state.db, request.app.state.config, req.video_id
1094:     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1102:     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
watermark)
1117:     req.video_id,
1129:     req.video_id,
1508:     video_id = compute_video_id(local_video)
1509:     was_reingest = db_instance.get_video(video_id) is not None
1521:     video_id, args.video_path, status, [r.model_dump() for r in val_results]
1550:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1553:     video_id, local_video, max_frames=args.max_frames

```



```
1567:         _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1572:         video_id=video_id,
```

7. user (2026-07-08T05:45:28.362Z)

```
240         _ingest_remote_segments,
241         _maybe_dispatch_remote,
242         _remote_input_size_bytes,
243         _report_config_for_input,
244         _resolve_compile,
245         read_path_rehydrate,
246         signed_reel_url,
247     )
248
249
250     # Legacy thin wrapper for any remaining direct calls during transition.
251     # New code should import load_app_config / AppConfig from backend.config directly.
252     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
253         cfg = load_app_config(config_path)
254         return cfg.model_dump(mode="json")
255
256
257     # --- API Models ---
258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269         compute: Optional[str] = None
270         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272         force: bool = False
273
274
275     class AnnotationsRequest(BaseModel):
276         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278         video_id: str
279         csv: str
280
281
282     class AssetRequest(BaseModel):
283         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285         #       local working copy for the run), OR
286         #   source_uri ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:/// URI) ?
287         #       registered as-is, no copy (D14b).
```

```

288         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
289         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290         uploaded_path: Optional[str] = None
291         source_uri: Optional[str] = None
292         medium_uri: str
293         sport: Optional[str] = None
294
295
296     class QueryRequest(BaseModel):
297         video_id: str
298         server: Optional[str] = None
299         receiver: Optional[str] = None
300         duration_min: Optional[float] = None
301         event_type: Optional[str] = None
302
303
304     class CompileRequest(BaseModel):
305         video_id: str
306         segment_ids: List[int]
307         output_filename: str
308         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
309         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
310         locale: Optional[str] = None
311
312
313     # --- API Endpoints ---
314     @app.get("/api/config")
315     def get_config(request: Request):
316         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
317         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
318         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
319         # by tests/test_redact.py + the /api/config redaction test).

```

8. user (2026-07-08T05:45:29.077Z)

```

904     @app.post("/api/process", status_code=202)
905     def process_video(req: ProcessRequest, request: Request):
906         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
907
908         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
909         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
910         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
911         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
912         """
913         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
914         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
915         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916         try:
917             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918         except edge_auth.EdgeAuthError as e:
919             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
920
921         allowed_root = getattr(
922             getattr(request.app.state.config, "security", None), "allowed_video_root", None
923         )

```

```

924         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926         allowed_uri_root = getattr(
927             getattr(request.app.state.config, "storage", None), "input_root", None
928         )
929         try:
930             validate_input_path(
931                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
932             )
933             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935             input_ref = storage.resolve_input_ref(
936                 req.video_path, getattr(request.app.state.config, "storage", None)
937             )
938             except ValueError as e:
939                 raise HTTPException(status_code=400, detail=str(e)) from e
940             # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941             # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942             # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943             # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
944             # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945             # rejects a non-local URI as out-of-root.
946             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947                 raise HTTPException(
948                     status_code=404, detail=f"Video file not found at '{req.video_path}'"
949                 )
950             # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
951             # the cache probe (#492 review): a cached gs:// result never fetches to local
scratch, so
952             # low temp space must not 507 it; a cache MISS still fails fast pre-claim exactly as
before.
953             if req.segmenter and not segmenter_registry.get(req.segmenter):
954                 available = list(segmenter_registry.list_available().keys())
955                 raise HTTPException(
956                     status_code=400,
957                     detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958                 )
959
960             # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
961             # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
962             # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
963             job_id = uuid.uuid4().hex
964             outcome = jobs_api.submit_process_job(
965                 request.app.state.db, request.app.state.config, job_id, req, owner_id, input_ref
966             )
967             if outcome is None:
968                 raise HTTPException(status_code=500, detail="Failed to persist job record.")
969             queued, response = outcome
970             if queued:
971                 _get_executor().submit(
972                     _drain_due_jobs, request.app.state.config, request.app.state.db
973                 )
974             return response

```

```

975
976
977     @app.get("/api/jobs/{job_id}/cost")
978     def get_job_cost(job_id: str, request: Request):
979         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
980         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
981         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
982         cost = request.app.state.db.get_job_cost(job_id)
983         if cost is None:
984             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
985         fx = request.app.state.config.billing.fx_inr_per_usd
986         usd = cost.get("cost_usd")
987         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
988         cost["fx_inr_per_usd"] = fx
989         return cost
990
991
992     @app.get("/api/videos/{video_id}/cost")
993     def get_video_cost(video_id: str, request: Request):

```

9. user (2026-07-08T05:45:38.746Z)

```

30:     db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
38:     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39:     the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
51:         video_id=video_id,
80: def submit_process_job(
86:     1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side effects.
92:         per video_id) and completes the job instantly: no drain, no GPU dispatch.
98:     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99:     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100:         db, cfg, video_id
104:     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
113:         db, cfg, video_id, req.video_path, requested_compute=req.compute
185:         "video_id": j.get("video_id"),

```

10. user (2026-07-08T05:45:43.949Z)

```

1     """Job-list endpoint (the processing-queue view) as its own router ? kept OUT of the
backend.main
2     god-module per the P24 route ceiling (see tests/test_main_module_budget.py; pattern:
backend/api/uploads.py).
3
4     GET /api/jobs is tenant-scoped and status-filtered so a client can discover in-flight
work it did NOT
5     submit (a page reload, a new tab, another device) and show its status ? which also stops
a tester who
6     can't see their running job from re-submitting the same video into a second, PAID GPU
run (#448). The
7     per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
8
9     Reads the live DB + config off ``request.app.state`` (set in backend.main.create_app),
so it needs no
10    module-global wiring and stays trivially testable via the standard
TestClient(create_app(cfg, db)).
11    """
12
13    import tempfile
14    from typing import Any, List, Optional, Tuple
15

```

```

16     from fastapi import APIRouter, HTTPException, Request
17     from fastapi.responses import JSONResponse
18
19     from backend.api import (
20         auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
21     )
22     from backend.utils.freespace import (
23         require_free_bytes, # HTTP-507 scratch guard (P2, D11)
24     )
25
26     router = APIRouter()
27
28
29     def claim_process_job(
30         db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31     ) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32         """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33         ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34         double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
35         ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
36         just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
37         ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
38         hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39         the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40         can re-associate the job with its bucket outputs (CP-rebuild $4.2); "" ? the drain
fills it."""
41         claimed = db.claim_or_create_process_job(
42             job_id,
43             req.video_path,
44             segmenter=req.segmenter,
45             player_pool=req.player_pool,
46             max_frames=req.max_frames,
47             owner_id=owner_id,
48             locale=req.locale,
49             compute=req.compute,
50             force=req.force,
51             video_id=video_id,
52         )
53         if claimed is None:
54             return None
55         if claimed["created"]:
56             return (True, None)
57         return (
58             False,
59             JSONResponse(
60                 status_code=202,
61                 content={
62                     "job_id": claimed["job_id"],
63                     "status": claimed["status"],
64                     "poll": f"/api/jobs/{claimed['job_id']}",
65                     "deduplicated": True,
66                 },
67             ),
68         )
69
70
71     def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:

```

```

72     """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
73     (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
copy."""
74     from backend import orchestration # call-time seam (mirrors submit_process_job)
75
76     need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
77     require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
78
79
80     def submit_process_job(
81         db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82     ) -> Optional[Tuple[bool, JSONResponse]]:
83         """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
one hook.
84
85         Order matters (#492 review):
86         1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
effects.
87         2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
fetches to local scratch, so low temp space must not reject it (review blocker
2).
88         3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
re-hydrate side effect, so the first request owns the cache completion (blocker
1).
89         4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
serialized
90         per video_id) and completes the job instantly: no drain, no GPU dispatch.
91
92         Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
``None`` ?
93
94         DB failure (the caller 500s)."""
95     from backend import orchestration # call-time seam: tests patch orchestration attrs
96
97     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
98     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
99         db, cfg, video_id
100     )
101
102     if input_ref.backend != "local" and not probed:
103         _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
104     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
105     if outcome is None:
106         return None
107     created, dedup_response = outcome
108     if not created:
109         assert dedup_response is not None # claim contract: not-created ? dedup
response
110         return (False, dedup_response)
111     if probed:
112         cached = orchestration.cached_process_result(
113             db, cfg, video_id, req.video_path, requested_compute=req.compute
114         )
115         if cached is not None:
116             db.mark_job_done(job_id, cached)
117             return (
118                 False,
119                 JSONResponse(
120                     status_code=202,
121                     content={
122                         "job_id": job_id,
123                         "status": "done",
124                         "poll": f"/api/jobs/{job_id}",
125                         "cached": True,
126                     },
127                 ),

```

```

128         )
129         # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
130         # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
131         # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
132         # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133         if input_ref.backend != "local":
134             try:
135                 _fetch_scratch_guard(cfg, input_ref)
136             except HTTPException:
137                 db.mark_job_failed(
138                     job_id,
139                     "cache re-hydrate failed and local scratch cannot hold a fetch of "
140                     "the source; free disk space or retry later",
141                 )
142             raise
143         return (
144             True,
145             JSONResponse(
146                 status_code=202,
147                 content={
148                     "job_id": job_id,
149                     "status": "queued",
150                     "poll": f"/api/jobs/{job_id}",
151                 },
152             ),
153         )
154
155     # Execution states a job can be in. ACTIVE = still doing (or about to do) work; the
queue view defaults
156     # to these. done/failed are terminal and only surface under ?status=all (or an explicit
single status).
157     ACTIVE_JOB_STATES = ("queued", "running", "waiting")
158     _LISTABLE_JOB_STATES = frozenset(ACTIVE_JOB_STATES) | {"done", "failed"}
159
160
161     @router.get("/api/jobs")
162     def list_jobs(request: Request, status: str = "active"):
163         """The caller's jobs (tenant-scoped), newest first. ``status=active`` (default) ?
164         queued|running|waiting; ``status=all`` ? recent jobs of any state; a single explicit
state is also
165         accepted (anything unknown degrades to the active queue ? never a raw 500). Light
projection: the
166         heavy per-job ``play_segments`` payload is dropped; ``result_status`` carries the
terminal verdict."""
167         try:
168             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
169         except edge_auth.EdgeAuthError as e:
170             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
171         if status == "all":
172             statuses: Optional[List[str]] = None
173         elif status in _LISTABLE_JOB_STATES:
174             statuses = [status]
175         else: # "active" and any unknown value ? the active queue
176             statuses = list(ACTIVE_JOB_STATES)
177         rows = request.app.state.db.list_jobs(owner_id, statuses=statuses, limit=50)
178         return {
179             "jobs": [
180                 {
181                     "job_id": j["id"],
182                     "kind": j.get("kind") or "process",
183                     "status": j["status"],
184                     "progress": j.get("progress"),

```

```

185         "video_id": j.get("video_id"),
186         "source": j.get("video_path"),
187         "compute": j.get("compute"),
188         "result_status": j.get("result_status"),
189         "error": j.get("error"),
190         "created_at": j.get("created_at"),
191         "started_at": j.get("started_at"),
192         "finished_at": j.get("finished_at"),
193     }
194     for j in rows
195 ]
196 }
197

```

11. user (2026-07-08T05:45:52.947Z)

```

16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
220:def submit_time_video_id(input_ref, cfg) -> Optional[str]:
287:def probe_process_cache(db, cfg, video_id: str) -> bool:
320:def cached_process_result(
751:     video_id = _main.compute_video_id(local_video)

```

12. user (2026-07-08T05:45:53.184Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8
9     from __future__ import annotations
10
11     import base64
12     import binascii
13     import logging
14     from datetime import timedelta
15     from importlib import import_module
16     from typing import Any, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat
20
21     logger = logging.getLogger(__name__)
22
23
24     def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25         """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26         is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27         have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28         if not md5_b64:
29             return None
30         try:
31             raw = base64.b64decode(md5_b64, validate=True)
32         except (binascii.Error, ValueError):
33             return None
34         return raw.hex() if len(raw) == 16 else None
35
36

```



```

37 class GCSStorage(StorageBackend):
38     def __init__(
39         self,
40         *,
41         client: Any = None,
42         project: Optional[str] = None,
43         signed_url_ttl: int = 3600,
44         **_ignored,
45     ) -> None:
46         self._ttl = signed_url_ttl
47         if client is not None:
48             self._client = client
49         return
50         storage_module: Any = import_module("google.cloud.storage")
51         self._client = storage_module.Client(project=project)
52
53     def _blob(self, ref: StorageRef):
54         return self._client.bucket(ref.bucket).blob(ref.key)
55
56     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57         if dst is None:
58             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59         import os
60
61         dst = str(dst)
62         if os.path.exists(dst) and not overwrite:
63             return dst
64         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
65         self._blob(ref).download_to_filename(dst)
66         return dst
67
68     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
69         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
70         return ref
71
72     def exists(self, ref) -> bool:
73         return bool(self._blob(ref).exists())
74
75     def stat(self, ref) -> StorageStat:
76         b = self._blob(ref)
77         b.reload()
78         return StorageStat(
79             size=int(b.size or 0),
80             modified=b.updated,
81             etag=b.etag,
82             content_type=b.content_type,
83             md5=_md5_hex(getattr(b, "md5_hash", None)),
84         )
85
86     def list(self, prefix) -> Iterator[StorageRef]:
87         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
88             yield StorageRef(
89                 "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
90             )
91
92     def delete(self, ref) -> None:
93         self._blob(ref).delete()
94
95     def open_writer(self, ref, *, content_type=None):
96         blob = self._blob(ref)
97         if content_type:
98             blob.content_type = content_type
99         # ignore_flush: BlobWriter only accepts flush() on chunk boundaries and raises
100         # otherwise; upload staging writes sequentially and finalizes with close(), so a
101         # caller-side flush must be a no-op, not an error. Bytes stream out per internal

```

```

102         # chunk (default 40 MiB) ? the whole object is never held in memory.
103         return blob.open("wb", ignore_flush=True)
104
105     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
106         blob = self._blob(ref)
107         exp = timedelta(seconds=ttl or self._ttl)
108         creds = getattr(self._client, "_credentials", None)
109         # Local-key signing (a service-account JSON key exposes a `signer`) works out of
the box. But
110         # on Cloud Run / GCE the ADC credentials come from the metadata server and have
NO private key,
111         # so a plain generate_signed_url raises "you need a private key". Fall back to
the IAM signBlob
112         # API ? which requires the runtime SA to hold
roles/iam.serviceAccountTokenCreator on ITSELF.
113         # ? signBlob canNOT ride the storage client's own token (#475, found on the
first real-creds
114         # run): that token is scoped to devstorage.* only (Client.SCOPE), which
iamcredentials rejects
115         # with ACCESS_TOKEN_SCOPE_INSUFFICIENT ? and a metadata credential may still
report the
116         # "default" email alias. Mint a separate cloud-platform-scoped credential
instead: its refresh
117         # both authorizes signBlob and resolves the real SA email. Failures are LOGGED
before deferring
118         # to the default call, so a broken sign path can't masquerade as a missing-key
error.
119         if creds is not None and not hasattr(creds, "signer"):
120             try:
121                 sa_email, token = self._signblob_identity()
122                 if sa_email:
123                     return blob.generate_signed_url(
124                         version="v4",
125                         expiration=exp,
126                         method=method,
127                         service_account_email=sa_email,
128                         access_token=token,
129                     )
130                 logger.warning(
131                     "[gcs] keyless signing skipped: runtime SA email unresolved (got
132                     %r); "
133                     "falling back to key-based signing",
134                     sa_email,
135                 )
136             except Exception as e:
137                 logger.warning(
138                     "[gcs] keyless signBlob path failed (%s); falling back to key-based
signing",
139                     e,
140                 )
141         return blob.generate_signed_url(version="v4", expiration=exp, method=method)
142
143     @staticmethod
144     def _signblob_identity() -> "tuple[Optional[str], Optional[str]]":
145         """(sa_email, access_token) for the IAM signBlob call ? a FRESH cloud-platform-
scoped
146         credential from ADC, never the storage client's devstorage-scoped token (#475).
Isolated
147         as a seam so tests can fake the metadata-server identity without real
credentials."""
148         import google.auth
149         from google.auth.transport.requests import Request as _AuthRequest
150
151         sign_creds, _ = google.auth.default(
            scopes=["https://www.googleapis.com/auth/cloud-platform"]

```

```

152         )
153         sign_creds.refresh(_AuthRequest())
154         sa_email = getattr(sign_creds, "service_account_email", None)
155         if not sa_email or sa_email == "default":
156             return None, None
157         return sa_email, sign_creds.token
158

```

13. user (2026-07-08T05:45:59.263Z)

```

200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
212         comes
213         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
214         "starting"
215         (queue/cold-start), so the very first tick already shows movement."""
216         minutes = int(elapsed_sec) // 60
217         if minutes < 1:
218             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
219         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
220
221     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
222         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
223         gs://
224         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
225         keep
226         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
227         no
228         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
229         if getattr(input_ref, "backend", "") != "gcs":
230             return None
231         sc = _storage_settings(cfg)
232         if not sc:
233             return None
234         try:
235             st = storage.get_storage("gcs", config=sc).stat(input_ref)
236             return getattr(st, "md5", None) or None
237         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
238             logger.info(
239                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
240                 getattr(input_ref, "uri", input_ref),
241                 exc_info=True,
242             )
243             return None
244
245     def _cached_process_payload(
246         video_id: str, validation_results, segments, requested_compute
247     ) -> Dict[str, Any]:
248         """The /api/process result for a video whose detection already exists ? ONE shape
249         for
250         both producers (the drain's DB-cache short-circuit and the submit-time cache,
251         #484)."""
252         return {

```

```

248         "video_id": video_id,
249         "status": "success",
250         "message": (
251             f"Reused {len(segments)} cached play segments. "
252             "Pass force=true to reprocess this video."
253         ),
254         "results": validation_results or [],
255         "play_segments": segments,
256         "cached": True,
257         "compute": {"path": "cached", "requested": requested_compute},
258     }
259
260
261 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
264     decide."""
265     ref = storage.parse_uri(uri)
266     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
267         local = storage.fetch(
268             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
269         )
270         with open(local, encoding="utf-8") as f:
271             loaded = json.load(f)
272         return loaded if isinstance(loaded, dict) else None
273
274 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
275 hits
276 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
277 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
278 instance
279 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
280 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
281 _REHYDRATE_LOCKS_GUARD = threading.Lock()
282
283 def _rehydrate_lock(video_id: str) -> threading.Lock:
284     with _REHYDRATE_LOCKS_GUARD:
285         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
286
287 def probe_process_cache(db, cfg, video_id: str) -> bool:
288     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
289     bucket)?
290     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
291     1
292     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
293     (blocker 2): a True probe means the request should never need CP scratch. Best-
294     effort:
295     errors report False (? the normal dispatch path with its guards)."""
296     try:
297         existing = db.get_video(video_id)
298         if (
299             existing
300             and existing.get("status") == "processed"
301             and db.query_play_segments(video_id, {})
302         ):
303             return True
304         sc = _storage_settings(cfg)
305         if not sc or (sc.get("backend") or "local") == "local":
306             return False
307         output_root = (sc.get("output_root") or "").rstrip("/")
308         if not output_root:

```

```

307         return False
308         report_uri = f"{output_root}/outputs/{video_id}/report.json"
309         if not storage.exists(report_uri, config=sc):
310             return False
311         report = _read_bucket_json(report_uri, sc)
312         return bool(report and report.get("status") == "success")
313     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314         logger.warning(
315             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316         )
317         return False
318
319
320     def cached_process_result(
321         db, cfg, video_id: str, video_path: str, requested_compute=None
322     ) -> Optional[Dict[str, Any]]:
323         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324         the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
325         re-ingests via `_ingest_remote_segments`): **the DB is a cache, the bucket is the
326         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). `None` ?
no
327         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328         branch mirrors the real cloud ingest exactly (watermarks + `_finalize_reingest`),
so a
329         partial earlier ingest is replaced, never double-counted.
330
331         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332         per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334         job before invoking (writes happen post-claim)."""
335         try:
336             with _rehydrate_lock(video_id):
337                 existing = db.get_video(video_id)
338                 if existing and existing.get("status") == "processed":
339                     segs = db.query_play_segments(video_id, {})
340                     if segs:
341                         return _cached_process_payload(
342                             video_id,
343                             existing.get("validation_results", []),
344                             segs,
345                             requested_compute,
346                         )
347                 sc = _storage_settings(cfg)
348                 if not sc or (sc.get("backend") or "local") == "local":
349                     return None
350                 output_root = (sc.get("output_root") or "").rstrip("/")
351                 if not output_root:
352                     return None
353                 outputs_uri = f"{output_root}/outputs/{video_id}"
354                 report_uri = f"{outputs_uri}/report.json"
355                 if not storage.exists(report_uri, config=sc):
356                     return None
357                 report = _read_bucket_json(report_uri, sc)
358                 if not report or report.get("status") != "success":
359                     return None
360                 if not video_path:
361                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362                     # carries the worker's --video-uri (P3); older reports fall back to the
363                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still

```

```

364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369         play_wm, cand_wm = db.segment_watermarks(video_id)
370         db.add_video(video_id, video_path, "processed", [])
371         n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372         if n <= 0:
373             return None
374         segments = [
375             s
376             for s in db.query_play_segments(video_id, {})
377             if int(s.get("id", 0)) > play_wm
378         ]
379         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

```

14. user (2026-07-08T05:46:07.203Z)

```

640         on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641     )
642     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
643         failure, not a 500
644         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
645         return _failed(f"cloud dispatch error: {e}", True)
646     if result.returncode != 0:
647         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
648     if result.video_id and result.video_id != video_id:
649         # The worker re-hashes the bucket bytes; a divergence means the input changed
650         between this
651         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
652         diagnosable.
653         logger.warning(
654             "[API] cloud video_id %s != local %s ? input bytes differ between the API
655             hash "
656             "and the worker hash; ingesting under the local id.",
657             result.video_id,
658             video_id,
659         )
660     notify("ingesting (cloud_run)")
661     storage_dict = (
662         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
663         can't narrow
664         if hasattr(storage_cfg, "model_dump")
665         else {}
666     )
667     try:
668         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
669     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
670         500 or leak rows
671         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
672         return _failed(
673             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
674             True,
675         )
676     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
677     drops the old ones.
678     segments = [
679         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
680     ]
681     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
682     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
683     is the real

```

```

678         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679         provenance = {
680             "path": "cloud_run",
681             "requested": req.compute,
682             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684             "execution": result.execution,
685             "outputs_uri": result.outputs_uri,
686         }
687         logger.info(
688             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689             result.execution,
690             provenance["job"],
691             provenance["region"],
692             result.outputs_uri,
693             video_id,
694         )
695         return (
696             {
697                 "video_id": video_id,
698                 "status": "success",
699                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
700                 "results": [r.model_dump() for r in val_results],
701                 "play_segments": segments,
702                 "compute": provenance,
703             },
704             True,
705         )
706
707
708     def _execute_processing(
709         config=None, db=None, req=None, progress=None
710     ) -> Dict[str, Any]:
711         """The full hash ? validate ? segment ? report pipeline for one video.
712
713         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714         module-level ``global_config``/``db_instance`` (backward-compatible with existing
715         tests that pass only ``req``). Route handlers always pass explicit params; older
716         test code that calls ``_execute_processing(req)`` still works.
717
718         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
719
720         Raises on unexpected internal errors ? the caller records those as a job failure
721         """
722         import backend.main as _main # call-time seam: main's globals/model stay patchable
723
724         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
725         # shift positional args ? req arrives in position 0 (config), progress in position 1
726         (db).
727         if config is not None and not isinstance(config, AppConfig):
728             # Old signature: _execute_processing(req, progress=None)
729             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
730                 req = db
731                 db = None
732             elif req is None and isinstance(config, _main.ProcessRequest):
733                 req = config
734                 config = None
735             # Resolve config/db from module globals when not passed explicitly
736             if config is None:
737                 config = _main.global_config
738             if db is None:
739                 db = _main.db_instance

```

```

738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
743     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
744     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
745     # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""
747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778     compute_actual: Optional[str] = None
779     response: Optional[Dict[str, Any]] = None
780     try:
781         if (
782             existing_video
783             and existing_video.get("status") == "processed"
784             and not req.force
785         ):
786             cached_segments = db.query_play_segments(video_id, {})
787             if cached_segments:
788                 logger.info(
789                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                     video_id,
791                 )
792             response = _cached_process_payload(
793                 video_id,
794                 existing_video.get("validation_results", []),
795                 cached_segments,

```



```

796         getattr(req, "compute", None),
797     )
798     return response
799
800     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801     #     UNLESS this exact content already has a cached verdict under the current
validator config.
802     notify("validating")
803     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808     # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809     # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810     # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811     # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812     # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813     # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814     val_fingerprint = registry.config_fingerprint(config)
815     # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816     # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817     # failing the job. The cache is a pure speedup ? never a new failure mode.
818     reused_validation = False
819     if not req.force:
820         try:
821             cached = db.get_validation_cache(video_id)
822             if cached is not None and cached.get("fingerprint") == val_fingerprint:
823                 # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824                 # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
825                 # deliberate cost of skipping the decode). Segmentation reads
neither of these.
826                 val_results = [
827                     ValidationResult(**r) for r in cached.get("results", [])
828                 ]
829                 reused_validation = True
830                 logger.info(
831                     "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832                     "validator config unchanged ? skipping the validator suite. "
833                     "force=true re-validates.",
834                     cached.get("passed"),
835                     video_id,
836                 )
837             except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838                 reused_validation = False
839                 logger.warning(

```

15. user (2026-07-08T05:46:20.396Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13
14     from __future__ import annotations
15
16     import os
17     import shutil
18     import tempfile
19     from typing import Any, Dict, Iterator, Optional
20
21     from backend.storage.base import StorageBackend
22     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24     __all__ = [
25         "StorageBackend",
26         "StorageRef",
27         "StorageStat",
28         "parse_uri",
29         "resolve_input_uri",
30         "get_storage",
31         "fetch",
32         "put",
33         "exists",
34         "signed_url",
35         "list_uris",
36         "open_writer",
37         "acquire_local_input",
38         "cleanup_acquired_local_input",
39         "input_is_local",
40         "resolve_input_ref",
41     ]
42
43
44     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
45         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
46         if storage_config is None:
47             return {}
48         if hasattr(storage_config, "model_dump"):
49             return storage_config.model_dump() # type: ignore[no-any-return]
50         return dict(storage_config)
51
52
53     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
54         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
55         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
56         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
```

```

57
58     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
59     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
60     sc = _storage_dict(storage_config)
61     uri = resolve_input_uri(
62         raw,
63         input_backend=sc.get("input_backend", "local"),
64         input_root=sc.get("input_root", ""),
65     )
66     return parse_uri(uri)
67
68
69     def input_is_local(raw: str, storage_config: Any = None) -> bool:
70         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
71         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
72         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
73         ``resolve_input_ref``)."""
74         return resolve_input_ref(raw, storage_config).backend == "local"
75
76
77     def acquire_local_input(
78         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
79     ) -> str:
80         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
81         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
82         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
83         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
84         path returned. The caller owns successful remote scratch copies; use
85         ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
86         sc = _storage_dict(storage_config)
87         ref = resolve_input_ref(raw, sc)
88         if ref.backend == "local":
89             return (
90                 ref.key
91             ) # identity ? no temp, no copy, byte-identical to passing the path directly
92         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
93         try:
94             return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
95         except Exception:
96             shutil.rmtree(scratch, ignore_errors=True)
97             raise
98
99
100
101     def cleanup_acquired_local_input(
102         raw: str, local_path: str, storage_config: Any = None
103     ) -> None:
104         """Delete a remote scratch dir produced by ``acquire_local_input``.
105         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
106         into a fresh ``rally-input-`` directory; deleting that directory is safe once the
caller has
107         finished hashing/validating/stitching/segmenting the returned file.
108         """
109         try:
110

```

```

111         if input_is_local(raw, storage_config):
112             return
113     except ValueError:
114         return
115     if not local_path:
116         return
117     scratch = os.path.dirname(os.path.abspath(local_path))
118     if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
119         shutil.rmtree(scratch, ignore_errors=True)
120
121
122 def _backend_class(name: str):
123     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
124     if name == "local":
125         from backend.storage.local import LocalStorage
126
127         return LocalStorage
128     if name == "s3":
129         from backend.storage.s3 import S3Storage
130
131         return S3Storage
132     if name == "gcs":
133         from backend.storage.gcs import GCSStorage
134
135         return GCSStorage
136     if name == "gdrive":
137         from backend.storage.gdrive import GDriveStorage
138
139         return GDriveStorage
140     if name == "gdrive-api":
141         from backend.storage.gdrive_api import GDriveApiStorage
142
143         return GDriveApiStorage
144     raise ValueError(
145         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146     )
147
148
149 def get_storage(
150     backend: Optional[str] = None,
151     *,
152     config: Optional[Dict[str, Any]] = None,
153     **overrides,
154 ) -> StorageBackend:
155     """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
156     selects the implementation; its per-backend settings come from ``config[backend]`` merged
157     with ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
158     selectors."""
159     cfg = config or {}
160     name = backend or cfg.get("backend", "local")
161     # Per-backend settings live under the backend name; tolerate the hyphen?underscore
162     # form so a
163     # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
164     # hyphen
165     # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
166     settings: Dict[str, Any] = dict(
167         cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
168     )
169     settings.update(overrides)
170     return _backend_class(name)(**settings)

```

```

171     uri: str,
172     dst: "str | os.PathLike[str] | None" = None,
173     *,
174     config: Optional[Dict[str, Any]] = None,
175     overwrite: bool = False,
176 ) -> str:
177     """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178     (no copy); for s3/gcs it downloads to ``dst``."""
179     ref = parse_uri(uri)
180     return get_storage(ref.backend, config=config).fetch_to_local(
181         ref, dst, overwrite=overwrite
182     )
183
184
185 def put(
186     local_path: "str | os.PathLike[str]",
187     uri: str,
188     *,
189     config: Optional[Dict[str, Any]] = None,
190     content_type: Optional[str] = None,
191 ) -> str:
192     """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193     ref = parse_uri(uri)
194     get_storage(ref.backend, config=config).put(
195         local_path, ref, content_type=content_type
196     )
197     return uri
198
199
200 def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
201     """True if the object at ``uri`` exists (a bucket object or a local file)."""
202     ref = parse_uri(uri)
203     return bool(get_storage(ref.backend, config=config).exists(ref))
204
205
206 def signed_url(
207     uri: str,
208     *,
209     config: Optional[Dict[str, Any]] = None,
210     ttl: Optional[int] = None,
211     method: str = "GET",
212 ) -> str:
213     """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
214     the app's response path (so a large object isn't streamed back through the server;
#463). Backends
215     without object signing (e.g. local) raise ``NotImplementedError``.
216
217     ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl``
from
218     ``config`` ? a hardcoded default here would silently shadow the operator's setting
(#469).
219     An explicit ``ttl`` overrides it for callers that need a specific expiry."""
220     ref = parse_uri(uri)
221     return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
222
223
224 def list_uris(
225     prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
226 ) -> Iterator[str]:
227     """Yield the URI of every object under ``prefix_uri``."""
228     ref = parse_uri(prefix_uri)
229     for r in get_storage(ref.backend, config=config).list(ref):

```

```

230         yield r.uri
231
232
233     def open_writer(
234         uri: str,
235         *,
236         config: Optional[Dict[str, Any]] = None,
237         content_type: Optional[str] = None,
238     ):
239         """A writable binary file-like streaming to ``uri`` (finalized on close). Backends
that
240         cannot stream raise ``NotImplementedError`` ? callers fall back to local staging
(see
241         ``StorageBackend.open_writer``)."""
242         ref = parse_uri(uri)
243         return get_storage(ref.backend, config=config).open_writer(
244             ref, content_type=content_type
245         )
246

```

16. user (2026-07-08T05:46:27.681Z)

```

88:         const response = await fetch("/api/process", {
125:             currentVideoId = data.video_id;
333:             video_id: currentVideoId,
380:             video_id: currentVideoId,

```

17. user (2026-07-08T05:46:34.965Z)

```

40         loadingSubtitle.textContent = subtitle;
41         loadingOverlay.style.display = "flex";
42     };
43
44     const hideLoader = () => {
45         loadingOverlay.style.display = "none";
46     };
47
48     const populatePlayerSelects = (players) => {
49         // Clear except first
50         queryServer.innerHTML = '<option value="">Any Server</option>';
51         queryReceiver.innerHTML = '<option value="">Any Receiver</option>';
52
53         players.forEach(player => {
54             const opt1 = document.createElement("option");
55             opt1.value = player;
56             opt1.textContent = player;
57             queryServer.appendChild(opt1);
58
59             const opt2 = document.createElement("option");
60             opt2.value = player;
61             opt2.textContent = player;
62             queryReceiver.appendChild(opt2);
63         });
64     };
65
66     // Initialize player selects at start
67     populatePlayerSelects(playerPool);
68
69     // --- 1. RUN VIDEO PROCESSING & VALIDATIONS ---
70     btnProcess.addEventListener("click", async () => {
71         const videoPath = videoPathInput.value.trim();
72         if (!videoPath) {
73             alert("Please enter a valid local MP4 video file path.");
74             return;
75         }

```

```

76
77 // Parse custom players
78 const rawPlayers = playerPoolInput.value.trim();
79 if (rawPlayers) {
80     playerPool = rawPlayers.split(",").map(p => p.trim()).filter(p => p.length >
81 0);
82     populatePlayerSelects(playerPool);
83 }
84 showLoader("Analyzing Video Stream", "Running structural diagnostics, video
focus checks, and relevance validators...");
85
86 try {
87     // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
88     const response = await fetch("/api/process", {
89         method: "POST",
90         headers: { "Content-Type": "application/json" },
91         body: JSON.stringify({
92             video_path: videoPath,
93             player_pool: playerPool
94         })
95     });
96
97     if (!response.ok) {
98         const err = await response.json();
99         throw new Error(err.detail || "Server failed to process the request.");
100     }
101
102     const submitted = await response.json();
103     const jobId = submitted.job_id;
104
105     // Poll until the job reaches a terminal state (done | failed).
106     let job;
107     for (;;) {
108         const jr = await fetch(`/api/jobs/${jobId}`);
109         if (!jr.ok) {
110             const err = await jr.json();
111             throw new Error(err.detail || "Failed to poll job state.");
112         }
113         job = await jr.json();
114         if (job.status === "done" || job.status === "failed") break;
115         showLoader("Analyzing Video Stream",
116             `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
117         await new Promise(res => setTimeout(res, 3000));
118     }
119
120     if (job.status === "failed") {
121         throw new Error(job.error || "Processing job failed.");
122     }
123
124     const data = job.result || {};
125     currentVideoId = data.video_id;
126
127     // Render validation list
128     renderValidationResults(data.results);
129
130     if (data.status === "failed") {
131         hideLoader();
132         alert("Validation Failed! This video does not meet our quality or
structural criteria. Please review the checklist.");
133         renderEmptyRallies("Validation failed. Please load a valid badminton raw
video.");
134         compileBoard.style.display = "none";
135         return;
136     }

```

```

137
138          // Successfully processed (field is play_segments ? the old `rallies` read
was a bug)
139          indexedRallies = data.play_segments || [];
140          selectedRallyIds.clear();
141          renderRallyCards(indexedRallies);
142
143          hideLoader();
144          alert(`Success! Successfully processed and indexed ${indexedRallies.length}
badminton rallies.`);
145
146      } catch (error) {
147          hideLoader();
148          alert(`Error: ${error.message}`);
149          renderEmptyRallies("Error processing file.");
150      }
151  });
152
153  // --- 2. RENDER VALIDATION CHECKLIST ---
154  const renderValidationResults = (results) => {
155      validationList.innerHTML = "";
156
157      results.forEach(res => {
158          const div = document.createElement("div");
159          div.className = `validation-item ${res.passed ? 'passed' : 'failed'}`;

```

18. user (2026-07-08T05:46:53.553Z)

Found 28 files

- backend\ui\assets\index-kJhH4dVE.js
- docs\CODE_MAP.md
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.test.tsx
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_upload_gcs_streaming.py
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_upload_download.py
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\docs\VIDEO_LOCALITY_MODEL.md
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\docs\CREATING_REELS.md
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\docs\CODE_MAP.md
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\ui\assets\index-503c-FCB.js
- C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\api\uploads.py
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts
- docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
- tests\test_upload_gcs_streaming.py
- backend\api\uploads.py
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.ts
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\upload.test.tsx
- tests\test_upload_download.py
- docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\mock-engine.mjs
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\contract-smoke.mjs
- docs\CREATING_REELS.md
- C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts
- docs\VIDEO_LOCALITY_MODEL.md

19. user (2026-07-08T05:46:59.273Z)

```

1      // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's

```



```

2      // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETES the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | {
18         status: "uploading";
19         pct: number;
20         filename: string;
21         sizeBytes: number;
22         sentBytes: number;
23         etaSec?: number;
24       }
25       | { status: "error"; error: IngestError };
26
27     export function useUpload(onComplete: (path: string, filename: string) => void,
maxUploadBytes?: number) {
28       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
29       const ctrlRef = useRef<AbortController | null>(null);
30       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
31       const onCompleteRef = useRef(onComplete);
32       onCompleteRef.current = onComplete;
33       // The engine's max upload size (BYTES) from GET /api/capabilities ? read through a
ref so `start`
34       // stays stable when the capability loads (same pattern as onCompleteRef). Absent/0 ?
no client guard.
35       const maxUploadBytesRef = useRef(maxUploadBytes);
36       maxUploadBytesRef.current = maxUploadBytes;
37
38       const stop = useCallback(() => {
39         ctrlRef.current?.abort();
40         ctrlRef.current = null;
41       }, []);
42
43       // Abort any in-flight upload on unmount.
44       useEffect(() => stop, [stop]);
45
46       const start = useCallback(
47         (file: File) => {
48           // A new pick supersedes any prior upload ? abort it FIRST, before the early-
return guards. If a
49           // rejected pick (0-byte / oversized) returned before stop(), an older in-flight
upload would keep
50           // running and could still call onComplete for the OLD file after we've shown an
error for the new
51           // one (processing A while the UI reports B was rejected).
52           stop();
53
54           // A 0-byte "video" is never valid ? reject before touching the network (reuse

```

```

invalidUri copy).
55         if (!file || file.size === 0) {
56             setPhase({ status: "error", error: { code: "invalidUri" } });
57             return;
58         }
59         // Oversized file: fail INSTANTLY (no wasted round-trip) ? /upload/init would 413
anyway. Carry
60         // the size + cap so IngestPanel can localize a "trim/chunk it first" message. No
cap ? skip the
61         // guard (the engine still fail-fasts server-side).
62         const cap = maxUploadBytesRef.current;
63         if (cap && file.size > cap) {
64             setPhase({ status: "error", error: { code: "fileTooLarge", sizeBytes: file.size,
capBytes: cap } });
65             return;
66         }
67
68         const ctrl = new AbortController();
69         ctrlRef.current = ctrl;
70         const cid = newCorrelationId();
71         const startedAtMs = Date.now();
72         setPhase({ status: "uploading", pct: 0, filename: file.name, sizeBytes: file.size,
sentBytes: 0 });
73         log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
74
75         void (async () => {
76             try {
77                 const res = await uploadFile(file, {
78                     cid,
79                     signal: ctrl.signal,
80                     onProgress: ({ sentBytes, totalBytes }) => {
81                         if (ctrl.signal.aborted) return;
82                         const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
83                         const elapsedSec = Math.max(0.25, (Date.now() - startedAtMs) / 1000);
84                         const rateBytesPerSec = sentBytes > 0 ? sentBytes / elapsedSec : 0;
85                         const remainingBytes = Math.max(0, totalBytes - sentBytes);
86                         const etaSec = rateBytesPerSec > 0 ? Math.ceil(remainingBytes /
rateBytesPerSec) : undefined;
87                         setPhase({
88                             status: "uploading",
89                             pct,
90                             filename: file.name,
91                             sizeBytes: totalBytes || file.size,
92                             sentBytes,
93                             etaSec,
94                         });
95                     },
96                 });
97                 if (ctrl.signal.aborted) return;
98                 setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
99                 onCompleteRef.current(res.path, file.name);
100             } catch (err) {
101                 // User-cancelled (abort) is silent, not an error.
102                 if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
103                 setPhase({ status: "error", error: classifyError(err) });
104                 log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
105             }
106         })();
107     },
108     [stop],
109 );

```

```

110
111     const cancel = useCallback(() => {
112         stop();
113         setPhase({ status: "idle" });
114         log("info", "upload.cancel", {});
115     }, [stop]);
116
117     const reset = useCallback(() => {
118         stop();
119         setPhase({ status: "idle" });
120     }, [stop]);
121
122     const busy = phase.status === "uploading";
123     return { phase, start, cancel, reset, busy };
124 }
125

```

20. user (2026-07-08T05:47:00.762Z)

```

87: * processing. POST /api/process {video_path} ? 202 {job_id, ...} [engine process route].
Fast-fails 400
100:   body: JSON.stringify({ video_path: videoPath, locale, compute }),
117:/** The source-video stream URL for annotation playback. GET /api/source/{video_id} (P8).
The engine
123:/** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
128:   body: JSON.stringify({ video_id: videoId, csv }),
134: * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
156:/** List a video's detected rallies. POST /api/query {video_id} ? {play_segments} [engine
query route]. */
160:   body: JSON.stringify({ video_id: videoId }),
190:   body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
226:/** The stream/download URL for a compiled reel. GET /api/highlights/{video_id}/{filename}
[engine highlights route]. */
248:// bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
291:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
312: * caller can classify it (the engine's 400/413 detail survives). Returns {path, video_id}.
337:   log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });

```

21. user (2026-07-08T05:47:08.399Z)

```

80
81     const data = (await res.json()) as T;
82     log("info", "api.response", { cid, method, url, status: res.status });
83     return data;
84 }
85
86 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
87 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [engine process
route]. Fast-fails 400
88 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
89 export async function processVideo(
90     videoPath: string,
91     cid?: string,
92     locale?: string,
93     compute?: string,
94 ): Promise<ProcessResponse> {
95     // `locale` (the UI language) is recorded for PMF analytics (which markets use the

```

```

tool). `compute`
96     // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
97     // default. Both optional + omitted when absent (JSON.stringify drops undefined).
98     return request<ProcessResponse>("/process", {
99         method: "POST",
100         body: JSON.stringify({ video_path: videoPath, locale, compute }),
101         cid,
102     });
103 }
104
105 /** Poll a submitted job. GET /api/jobs/{job_id} [engine jobs route]. 404 ? unknown
job_id. */
106 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
108 }
109
110 /** List indexed videos, newest first ? the "my videos" library. GET /api/videos [engine
list_videos route].
111 * Reload-survival: a past match can be reopened without re-processing. Returns [] on
an empty box. */
112 export async function listVideos(cid?: string): Promise<VideoSummary[]> {
113     const data = await request<VideosResponse>("/videos", { method: "GET", cid });
114     return data.videos ?? [];
115 }
116
117 /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
(P8). The engine
118 * serves a frame-identical proxy; labels stay keyed to the master's MD5
(STUDIO_MEDIA_LIFECYCLE D4). */
119 export function sourceUrl(videoId: string): string {
120     return `${BASE}/source/${encodeURIComponent(videoId)}`;
121 }
122
123 /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
124 * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
into the corpus. */
125 export async function postAnnotations(videoId: string, csv: string, cid?: string):
Promise<AnnotationsResponse> {
126     return request<AnnotationsResponse>("/annotations", {
127         method: "POST",
128         body: JSON.stringify({ video_id: videoId, csv }),
129         cid,
130     });
131 }
132
133 /** Store a video into the user's chosen medium + register it as a library asset (P4,
D14).
134 * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
135 * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
MD5-keyed copy is
136 * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
video already
137 * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
(bad medium /
138 * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
502 (put) ?
139 * surfaced as ApiError. */
140 export async function createAsset(
141     args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
},
142     cid?: string,

```



```

286     }
287     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
288     return (await res.json()) as UploadPartResponse;
289 }
290
291 /** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
292 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
293     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
294         method: "POST",
295         body: JSON.stringify({ total_parts: totalParts }),
296         cid,
297     });
298 }
299
300 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[backend/api/uploads.py]. */
301 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
302     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
303 }
304
305 export interface UploadProgress {
306     sentBytes: number;
307     totalBytes: number;
308 }
309
310 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
311 * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
312 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
313 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
314 * fresh init (re-pick the file), which the UI's retry does. */
315 export async function uploadFile(
316     file: File,
317     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
318 ): Promise<UploadCompleteResponse> {
319     const cid = opts.cid ?? newCorrelationId();
320     const init = await uploadInit(file.name, file.size, cid);
321     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
322     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
323     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
324     const partBytes = partMb * 1024 * 1024;
325     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
326     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
327     let sent = 0;
328     try {
329         for (let i = 0; i < totalParts; i++) {
330             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
331             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
332             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
333             sent += chunk.size;
334             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
335         }
336         const done = await uploadComplete(init.upload_id, totalParts, cid);
337         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:

```

```

done.size_bytes });
338     return done;
339 } catch (err) {
340     log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
341     await uploadAbort(init.upload_id, cid).catch(() => {});
342     throw err;
343 }
344 }
345

```

23. assistant (2026-07-08T05:47:27.965Z)

The SPA discards `video\_id` (logs only) and hands `path` to the process submit. Let me verify what IngestPanel does with `onComplete`, then map the drain/worker side.

24. user (2026-07-08T05:47:29.891Z)

```

13:import { useUpload } from "../hooks/useUpload";
149:     submit(asset.stored_uri, locale, sendCompute, displayName ??
sourceLabel(asset.stored_uri) ?? undefined); // index the stored copy
160: // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
161: const upload = useUpload((path, filename) => {
162:     if (medium === "local") submit(path, locale, sendCompute, filename);
185:     submit(uri, locale, sendCompute, displayName ?? undefined);

```

25. user (2026-07-08T05:47:30.721Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18
19    import json
20    import logging
21    import os
22    import threading
23    import traceback
24    from concurrent.futures import ThreadPoolExecutor
25    from typing import Any, Dict, Optional
26
27    logger = logging.getLogger(__name__)
28
29    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32    # already parallelize their AI handoff internally via their own pools.

```

```

33     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
    _cloud_dispatch_possible), a REMOTE
34     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
    Gemini), so the drain
35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
    parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
    invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
    _get_executor() creation.
39     #
40     # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
    configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
    call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
    override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
    dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
    parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
    _cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
    dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
    and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
    count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
    jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
    absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)

```



```

82         return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83     return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec``,
106         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
107         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
108         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
109
110
111         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
112         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
113         """
114
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
the original submit. The row stores exactly the ProcessRequest fields."""
125         import backend.main as _main
126
127         raw_params = job.get("params")
128         params = (
129             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
130         )
131         return _main.ProcessRequest(
132             video_path=job["video_path"],
133             player_pool=job.get("player_pool"),
134             max_frames=job.get("max_frames"),
135             segmenter=job.get("segmenter"),
136             # v8 compute-picker: the worker runs _execute_processing ?
137             _maybe_dispatch_remote, which
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is

```

```

only read off
139         # the job row for analytics). NULL ? server default = pre-picker behaviour.
140         compute=job.get("compute"),
141         force=bool(params.get("force")),
142     )
143
144
145 def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152     job self-scheduled and will be re-claimed when `next_poll_at` is due.
153     """
154     import backend.main as _main
155
156     job_id = job["id"]
157     # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158     # original /api/process pipeline). Both record their terminal state the same way;
only process
159     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160     is_compile = job.get("kind") == "compile"
161     try:
162         if is_compile:
163             result = _main._execute_compile(
164                 config,
165                 db,
166                 job,
167                 progress=lambda stage: db.set_job_progress(job_id, stage),
168             )
169         else:
170             req = _main._job_to_request(job)
171             result = _main._execute_processing(
172                 config,
173                 db,
174                 req,
175                 progress=lambda stage: db.set_job_progress(job_id, stage),
176             )
177             if result.get("video_id"):
178                 db.set_job_video_id(job_id, result["video_id"])
179             db.mark_job_done(job_id, result)
180             if not is_compile:
181                 _maybe_record_job_cost(
182                     job, result, config, db
183                 ) # M8 cost ledger (default-OFF; process jobs only)
184                 _maybe_run_flywheel(
185                     job, result, config
186                 ) # M11 data-flywheel (process jobs only)
187     except _main.JobWaiting as w:
188         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191     except Exception as e:
192         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196 def _maybe_record_job_cost(

```

```

197     job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198 ) -> None:
199     """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200     ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201     (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
202     persist it.
203
204     ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
205     ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
206     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
207     + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
208     With the placeholder pricebook the cost is 0 regardless.
209
210     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
211     claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
212
213     billing_cfg = getattr(config, "billing", None)
214     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
215         return
216     try:
217         usage = (result or {}).get("usage") or {}
218         db.record_job_cost(
219             job["id"],
220             usage=usage,
221             pricebook_version=billing_cfg.pricebook_version,
222             gpu_type=usage.get("gpu_type"),
223             compute_backend=(result or {}).get("compute_backend"),
224             owner_id=job.get("owner_id"),
225             margin=billing_cfg.margin,
226         )
227         logger.info(
228             "Recorded cost for job %s (pricebook=%s).",
229             job["id"],
230             billing_cfg.pricebook_version,
231         )
232     except Exception as e: # never wedge a completed job on bookkeeping
233         logger.warning(
234             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235         )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250         flywheel_cfg = getattr(config, "flywheel", None)

```

```

251     if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252         return
253     try:
254         from backend import flywheel
255
256         flywheel.run_for_job(job, result, config)
257     except Exception as e: # never wedge a completed job on the flywheel
258         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261 def _drain_due_jobs(config: Any, db: Any) -> int:
262     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267     a single-worker box keeps making progress with no central timer (the DB is the
clock).
268     Returns the number of jobs that reached a terminal/parked state this tick.
269     """
270     import backend.main as _main
271
272     ran = 0
273     while True:
274         job = db.claim_due_job()
275         if job is None:
276             break
277         ran += 1
278         _main._run_claimed_job(job, config, db)
279     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281     # on the next submit/drain. Best-effort ? never raises into the worker.
282     _main._schedule_next_drain_if_waiting(config, db)
283     return ran
284
285
286 def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288     self-scheduled job resumes with no further client submit. The drain itself re-checks
289     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290     import backend.main as _main
291
292     try:
293         delay = db.seconds_until_next_due()
294     except Exception as e: # pragma: no cover - defensive; never wedge the worker
295         logger.error(f"Could not compute next-due delay: {e}")
296         return
297     if delay is None:
298         return # nothing waiting
299     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303 def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can

```

```

306 monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting. """
307 import backend.main as _main
308
309 timer = threading.Timer(
310     delay_sec,
311     lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312 )
313 timer.daemon = True
314 timer.start()
315
316
317 #: A parked job further out than this is re-checked by a capped wake rather than one
long
318 #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319 _MAX_DRAIN_WAKE_SEC = 60.0
320
321
322 #
-----
323 # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324 # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
325 # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326 # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327 # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
328 # re-dispatching (no double GPU spend).
329
330 #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331 _RESUME_POLL_EVERY_SEC = 15.0
332
333
334 def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335     """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336     orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
337     after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338     thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339     NEVER re-dispatched from here. Returns the number of resumes started.
340
341     On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342     exist ? the rows get today's honest terminal failure instead. """
343     rows = db.interrupted_remote_process_jobs()
344     if not rows:
345         return 0
346     if not _cloud_dispatch_possible(config):
347         for job in rows:
348             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349             logger.warning(
350                 "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                 "dispatch to Cloud Run, so no remote worker can be in flight.",
352                 len(rows),
353             )
354     return 0
355
356 # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
357 # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
# default target itself is cloud_run (on a default-local server with cloud-override

```

```

358     # capability, a NULL row ran in-process ? no remote worker exists to resume).
359     default_is_cloud = (
360         getattr(getattr(config, "deployment", None), "compute_target", "") ==
361         "cloud_run"
362     )
363     started = 0
364     for job in rows:
365         # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
366         # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
367         requested = (str(job.get("compute") or "")).strip().lower()
368         if requested != "cloud" and not default_is_cloud:
369             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
370             continue
371         t = threading.Thread(
372             target=_resume_one_remote_job,
373             args=(config, db, dict(job)),
374             daemon=True,
375             name=f"resume-{str(job['id'])[:8]}",
376         )
377         t.start()
378         started += 1
379     if started:
380         logger.warning(
381             "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
382             started,
383         )
384     return started
385
386 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389     fail the job HONESTLY from what the bucket says. Never dispatches."""
390     from backend import orchestration, storage
391     from backend.infrastructure.execution_policy import (
392         ExecutionPolicy,
393         PolicyTimeout,
394         poll_until,
395     )
396
397     job_id = str(job["id"])
398     video_id = str(job["video_id"])
399     try:
400         raw_params = job.get("params")
401         params = (
402             raw_params
403             if isinstance(raw_params, dict)
404             else json.loads(raw_params or "{}")
405         )
406         if params.get("force"):
407             # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408             # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409             # bypasses it, and nothing proves the report belongs to the forced run
rather
410             # than an older completed one.
411             db.mark_job_failed(
412                 job_id,
413                 "control plane restarted during a forced reprocess; the cached result "

```

```

414         "cannot be trusted for force=true ? re-submit with force",
415     )
416     return
417 if (str(job.get("compute") or "").strip().lower() == "local":
418     # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419     # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420     db.mark_job_failed(job_id, "interrupted by server restart")
421     return
422 sc = orchestration._storage_settings(config)
423 output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424 if not sc or not output_root:
425     db.mark_job_failed(
426         job_id,
427         "interrupted by server restart (no output bucket configured to recover
from)",
428     )
429     return
430 report_uri = f"{output_root}/outputs/{video_id}/report.json"
431 max_wait = float(
432     getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433     or 3900.0
434 )
435 policy = ExecutionPolicy(
436     poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437     op_timeout_sec=120.0,
438     max_wait_sec=max_wait,
439 )
440 db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441 try:
442     poll_until(
443         lambda: True if storage.exists(report_uri, config=sc) else None,
444         policy,
445         label=f"resume-{job_id[:8]}",
446         logger=logger,
447         # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448         on_tick=lambda elapsed: db.set_job_progress(
449             job_id,
450             "recovering: waiting for the in-flight cloud run ? "
451             f"{int(elapsed) // 60} min",
452         ),
453     )
454 except PolicyTimeout:
455     db.mark_job_failed(
456         job_id,
457         "control plane restarted mid-run and the cloud worker's result never "
458         "appeared within the poll budget; re-submit the video (a finished result
"
459         "re-hydrates instantly)",
460     )
461     return
462 payload = orchestration.cached_process_result(
463     db, config, video_id, str(job.get("video_path") or "")
464 )
465 if payload is not None:
466     db.mark_job_done(job_id, payload)
467     logger.warning(
468         "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469         job_id,
470         video_id,
471     )
472     return

```

```

473         # The report exists but did not realize into a usable result ? surface its
verdict.
474         status = "unreadable"
475         try:
476             report = orchestration._read_bucket_json(report_uri, sc) or {}
477             status = str(report.get("status")) or "unreadable"
478         except Exception: # noqa: BLE001 ? the failure message is best-effort
479             pass
480         db.mark_job_failed(
481             job_id,
482             f"cloud worker finished without a usable result after a control-plane
restart "
483             f"(report status: {status}); re-submit the video",
484         )
485     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487         try:
488             db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489         except Exception: # noqa: BLE001
490             pass
491

```

26. user (2026-07-08T05:47:39.328Z)

```

380         if not segments:
381             return None
382         logger.info(
383             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384             n,
385             video_id,
386             outputs_uri,
387         )
388         return _cached_process_payload(video_id, [], segments, requested_compute)
389     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390         logger.warning(
391             "[API] submit-time cache check failed for %s ? dispatching normally",
392             video_id,
393             exc_info=True,
394         )
395         return None
396
397
398     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400         try:
401             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403             return ""
404
405
406     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/` ` ?
so
412         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
413         has the video's rows now (re-query). Non-md5 ids return False without touching the
bucket; every failure degrades to False (the read just reports what the DB has)."""
414         vid = (video_id or "").strip().lower()
415

```



```

416         if not _MD5_HEX_RE.fullmatch(vid):
417             return False
418         if not probe_process_cache(db, cfg, vid):
419             return False
420         return cached_process_result(db, cfg, vid, "") is not None
421
422
423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435         ``deployment.compute_target
436         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437         intervals into
438         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439
440         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441         the cloud job
442         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443         observability report
444         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445         process segmenter
446         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
447         non-``cloud_run`` target, so
448         the in-process path stays byte-identical)."""
449         from backend.compute import ComputeJobSpec, get_compute_backend
450
451     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453         (id > play_wm);
454         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
455         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
456         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
457         only when the
458         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
459         rc / ingest
460         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
461         records the
462         # intended backend, `requested` the picker choice, `dispatched` whether the
463         remote job ran.
464         return (
465             {
466                 "video_id": video_id,
467                 "status": "failed",
468                 "message": msg,
469                 "results": [r.model_dump() for r in val_results],
470                 "play_segments": [],
471                 "compute": {
472                     "path": "cloud_run" if ran else "not_run",
473                     "requested": req.compute,
474                     "target": "cloud_run",
475                     "dispatched": ran,
476                 },
477             },
478             ran,
479         )
480

```

```

470         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473         choice = (req.compute or "").strip().lower()
474         if choice and choice not in ("local", "cloud"):
475             logger.warning(
476                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477                 req.compute,
478             )
479             choice = ""
480         if choice == "local":
481             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487             # below exactly as before (the weights-present check is the escape hatch).
488             if (
489                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490                 and _cloud_available(cfg)
491                 and not _local_detector_weights_present(cfg, seg_name)
492             ):
493                 return _failed(
494                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                     "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496                     False,
497                 )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499             if choice == "cloud":
500                 if not _cloud_available(cfg):
501                     return _failed(
502                         "cloud-serving was requested but this server isn't configured for it "
503                         "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                         False,
505                     )
506                 compute = get_compute_backend(cfg, target_override="cloud_run")
507             else:
508                 compute = get_compute_backend(
509                     cfg
510                 ) # server default (default-OFF unless cloud_run)
511             if compute is None:
512                 return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514             storage_cfg = getattr(cfg, "storage", None)
515
516             # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517             try:
518                 input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519             except ValueError as e:

```

```

520         return _failed(
521             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522         )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )
560         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561             logger.error(
562                 "[API] cloud-serving: failed to stage %s -> %s: %s",
563                 local_src,
564                 staged_uri,
565                 e,
566                 exc_info=True,
567             )
568             return _failed(
569                 f"cloud-serving: failed to stage the uploaded file to {input_root}:

```

```

{e}",
570             False,
571         )
572         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575         req.video_path = staged_uri
576         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577         db.add_video(
578             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579         )
580         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581         if not out_root:
582             return _failed(
583                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584                 "? that's where the Cloud Run job writes the result.",
585                 False,
586             )
587
588         notify("dispatching (cloud_run)")
589         logger.info(
590             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591             req.video_path,
592             out_root,
593         )
594         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598         overrides = []
599         idx_cfg = getattr(cfg, "indexing", None)
600         sk = getattr(idx_cfg, "skip_ai_handoff", None)
601         if sk is not None:
602             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:
611             decode_backend = getattr(wasb_cfg, "decode_backend", None)
612             if decode_backend:
613                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614             batch_size = getattr(wasb_cfg, "batch_size", None)
615             if batch_size:
616                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
617             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618             if prefetch:
619                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")

```

```

620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,

```

27. user (2026-07-08T05:47:47.486Z)

```

840             "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841             video_id,
842             exc_info=True,
843         )
844         if not reused_validation:
845             logger.info(f"Running validations for {req.video_path}...")
846             val_results, val_context = registry.run_all_with_context(local_video,
config)
847             failures = [r for r in val_results if not r.passed]
848             if not reused_validation:
849                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
850                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
851                 db.set_validation_cache(
852                     video_id,
853                     val_fingerprint,
854                     not failures,
855                     [r.model_dump() for r in val_results],
856                 )
857
858             if failures:
859                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
860                 # status; it no longer destroys the video's prior good segments.
861                 db.add_video(
862                     video_id,
863                     req.video_path,
864                     "failed",
865                     [r.model_dump() for r in val_results],
866                 )
867             response = {
868                 "video_id": video_id,
869                 "status": "failed",
870                 "message": _validation_failure_message(failures),

```

```

871         "results": [r.model_dump() for r in val_results],
872     }
873     return response
874
875     # 2. Run segmenter & indexer
876     logger.info("[API] Video passed checks. Segmenting and indexing...")
877     db.add_video(
878         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
879     )
880
881     segmenter_cls = segmenter_registry.get(seg_name)
882     if not segmenter_cls:
883         # Submit-time validation already 400s unknown names; this is the defensive
884         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
885         available = list(segmenter_registry.list_available().keys())
886         response = {
887             "video_id": video_id,
888             "status": "failed",
889             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
890             "results": [r.model_dump() for r in val_results],
891             "play_segments": [],
892         }
893         return response
894
895     logger.info(f"[API] Using segmenter: {seg_name}")
896     notify(f"segmenting ({seg_name})")
897     segmenter_actual = seg_name
898     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
899     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
900     play_wm, cand_wm = db.segment_watermarks(video_id)
901     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
902     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
903     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
904     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
905     # they're dropped on the cloud path in this slice ? a documented follow-up.)
906     remote = _maybe_dispatch_remote(
907         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
908     )
909     if remote is not None:
910         response, segmentation_ran = (
911             remote # `ran` = did the cloud job actually execute (report)
912         )
913         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
914     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
915     compute_actual = "in_process"
916     segmenter = segmenter_cls(db, config)
917     try:
918         # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
919         # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
920         with _LOCAL_COMPUTE_LANE:
921             result = segmenter.process_video(
922                 video_id, local_video, req.player_pool, req.max_frames
923             )

```

```

924     except Exception:
925         # A raising segmenter must not leave half-written rows: restore the pre-run
926         # state (same destroy-after-confirm contract as the Failure branch), then
927         let
928             # the job worker record the failure.
929             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
930             raise
931     segmentation_ran = True
932
933     if isinstance(result, Failure):
934         logger.error(f"[API] Segmenter failed: {result.message}")
935         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
936         response = {
937             "video_id": video_id,
938             "status": "failed",
939             "message": f"Segmenter '{seg_name}' failed: {result.message}",
940             "results": [r.model_dump() for r in val_results],
941             "play_segments": [],
942         }
943         return response
944
945     segments = result.value if hasattr(result, "value") else result
946     notify("finalizing")
947     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
948
949     response = {
950         "video_id": video_id,
951         "status": "success",
952         "message": f"Successfully indexed {len(segments)} play segments using
953         '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
954         .value
955         "results": [r.model_dump() for r in val_results],
956         "play_segments": segments,
957     }
958     return response
959
960     finally:
961         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
962         already set
963         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
964         other path stamp
965         # the actual path here so the job result PROVES which backend ran (in_process /
966         not_run) and
967         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
968         then visible.
969         if isinstance(response, dict):
970             response.setdefault(
971                 "compute",
972                 {
973                     "path": compute_actual or "not_run",
974                     "requested": getattr(req, "compute", None),
975                 },
976             )
977         prov = response.get("compute", {}) if isinstance(response, dict) else {}
978         logger.info(
979             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
980             video_id,
981             getattr(req, "compute", None),
982             prov.get("path"),
983             prov.get("dispatched"),
984             (response or {}).get("status") if isinstance(response, dict) else None,
985         )
986         # Always emit the per-video observability report (next to the video, or to
987         # report.output_dir). Never raises. Surface the paths in the response. (The
988         compute-path
989         # proof lives on the job result + the [COMPUTE] log, not the report ?

```

```

obsreport's record_run
981         # doesn't surface arbitrary run signals.)
982         paths = _main.emit_indexer_report(
983             db=db,
984             video_id=video_id,
985             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
986             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
987             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
988             # still preserved in DB provenance via db.add_video.
989             video_path=local_video,
990             config=_report_config_for_input(config, req.video_path),
991             val_results=val_results,
992             val_context=val_context,
993             segmenter_requested=req.segmenter,
994             segmenter_actual=segmenter_actual,
995             was_reingest=was_reingest,
996             segmentation_ran=segmentation_ran,
997         )
998         if paths and isinstance(response, dict):
999             response["reports"] = paths

```

28. user (2026-07-08T05:48:01.385Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11        job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15        Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16        worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17        (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21        The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22        so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23

```



```

24     The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
25     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26     owner-verified on the M7 real run.
27     """
28
29     from __future__ import annotations
30
31     import json
32     import logging
33     import os
34     import tempfile
35     import uuid
36     from abc import ABC, abstractmethod
37     from typing import Any, Callable, List, Optional
38
39     from backend.compute.base import (
40         ComputeBackend,
41         ComputeJobSpec,
42         ComputeResult,
43         RemoteExecutionFailed,
44     )
45     from backend.infrastructure.execution_policy import (
46         DEFAULT_POLICY,
47         ExecutionPolicy,
48         PolicyTimeout,
49         poll_until,
50     )
51     from backend.storage import fetch, get_storage, parse_uri
52
53     LOG = logging.getLogger("compute.cloud_run")
54
55     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58         "deployment.compute_target=local_gpu",
59         "indexing.detector_impl=native",
60     )
61
62
63     class CloudRunJobClient(ABC):
64         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
65
66         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67         overrides the job's container ``args`` for this execution and starts it."""
68
69         @abstractmethod
70         def run_job(
71             self,
72             job_name: str,
73             argv: List[str],
74             *,
75             region: str,
76             project: Optional[str] = None,
77         ) -> str:
78             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80

```

```

81         @abstractmethod
82         def cancel_execution(self, execution: str) -> None:
83             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86             NEVER masked by a cancel error)."""
87
88         def execution_terminal_state(self, execution: str) -> Optional[str]:
89             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91             can't be determined).
92
93             ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94             terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95             of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97             NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98             chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104         class GoogleCloudRunJobClient(CloudRunJobClient):
105             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108             def run_job(
109                 self,
110                 job_name: str,
111                 argv: List[str],
112                 *,
113                 region: str,
114                 project: Optional[str] = None,
115             ) -> str:
116                 # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117                 # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118                 # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119                 if not project:
120                     raise RuntimeError(

```

29. user (2026-07-08T05:48:02.077Z)

```

1430         # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1431         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1432         if args.output_dir is not None:
1433             global_config.output.output_dir = resolve_output_dir(args.output_dir)

```

```

1434     else:
1435         global_config.output.output_dir = resolve_output_dir(
1436             global_config.output.output_dir
1437         )
1438     os.makedirs(global_config.output.output_dir, exist_ok=True)
1439
1440     db_path = os.path.join(
1441         global_config.output.output_dir, global_config.output.db_name
1442     )
1443
1444     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1445     # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1446     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1447     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1448     global _instance_lock
1449     try:
1450         _instance_lock = acquire_lock(db_path + ".lock")
1451     except OSError as exc:
1452         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1453         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1454         # legitimate single instance).
1455         logger.warning(
1456             "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
1457             "WITHOUT the second-instance guard.",
1458             db_path,
1459             exc,
1460         )
1461         _instance_lock = None
1462     else:
1463         if _instance_lock is None:
1464             print(
1465                 f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1466                 f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
1467                 f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
1468             )
1469             sys.exit(1)
1470
1471     db_instance = Database(db_path)
1472
1473     # Crash recovery (#16, #329): jobs left queued/running by a previous process are
dead ?
1474     # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so
a reel
1475     # interrupted by a Cloud Run scale-down finishes on the next drain instead of
failing the
1476     # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal
state.
1477     requeued, failed = db_instance.recover_stale_jobs()
1478     if requeued or failed:
1479         logger.warning(
1480             f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
1481             f"{failed} process failed."
1482         )
1483     # P3 (#471): md5-keyed RUNNING remote jobs resume on outputs/<md5>/report.json ?

```

```

never re-dispatched.
1484     resume_interrupted_remote_jobs(global_config, db_instance)
1485
1486     # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1487     # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1488     # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1489     _maybe_sweep_gemini_orphans(global_config)
1490
1491     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1492     if args.video_path and not args.server and not args.app:
1493         print(f"[CLI] Processing video file: {args.video_path}")
1494         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1495         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1496         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1497         storage_cfg = getattr(global_config, "storage", None)
1498         try:
1499             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1500         except ValueError as e:
1501             print(f"[FAIL] {e}")
1502             return
1503         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1504             print(f"[FAIL] Video file not found: {args.video_path}")
1505             return
1506         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1507         try:
1508             video_id = compute_video_id(local_video)
1509             was_reingest = db_instance.get_video(video_id) is not None
1510
1511             # Validate (with-context variant surfaces ffprobe_meta for the report)
1512             print("[CLI] Validating...")
1513             val_results, val_context = registry.run_all_with_context(
1514                 local_video, global_config
1515             )
1516             failures = [r for r in val_results if not r.passed]
1517
1518             # Save video status
1519             status = "failed" if failures else "processed"
1520             db_instance.add_video(
1521                 video_id, args.video_path, status, [r.model_dump() for r in val_results]
1522             )
1523
1524             seg_name = args.segmenter or getattr(
1525                 getattr(global_config, "indexing", None), "default_segmenter", "motion"
1526             )
1527             segmenter_actual = None
1528             segmentation_ran = False
1529             try:
1530                 print("\n" + "=" * 40)
1531                 print("VALIDATION SUMMARY")
1532                 print("=" * 40)
1533                 for r in val_results:
1534                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535                     print(f"{status_symbol} {r.validator_name}: {r.message}")
1536                 print("=" * 40 + "\n")
1537
1538             if failures:
1539                 print("[FAIL] Video failed checks. Indexing halted.")
1540                 return

```

```

1541
1542         # Index
1543         segmenter_cls = segmenter_registry.get(seg_name)
1544         if not segmenter_cls:
1545             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546             return
1547
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True
1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565             )
1566             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568         finally:
1569             # Always emit the per-video observability report. Never raises.
1570             paths = emit_indexer_report(
1571                 db=db_instance,
1572                 video_id=video_id,
1573                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                 # in DB provenance via db.add_video.
1576                 video_path=local_video,
1577                 config=_report_config_for_input(global_config, args.video_path),
1578                 val_results=val_results,
1579                 val_context=val_context,
1580                 segmenter_requested=args.segmenter,
1581                 segmenter_actual=segmenter_actual,
1582                 was_reingest=was_reingest,
1583                 segmentation_ran=segmentation_ran,
1584             )
1585             if paths:
1586                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589         finally:
1590             storage.cleanup_acquired_local_input(
1591                 args.video_path, local_video, storage_cfg
1592             )
1593         return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.

```

30. user (2026-07-08T05:48:10.111Z)

No files found

31. user (2026-07-08T05:48:12.004Z)

```
380:class SecurityConfig(BaseModel):
383:    allowed_video_root: Optional[str] = None
387:    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
388:    # contain upload_dir or /api/process will reject the uploaded paths.
389:    upload_dir: str = "output/uploads"
392:    max_upload_mb: int = 4096
393:    max_part_mb: int = 95
394:    allowed_upload_extensions: List[str] = ["mp4", "mov"]
413:class StorageConfig(BaseModel):
423:    output_root: str = ""
424:    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
427:    # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
428:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
432:    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433:    input_root: str = ""
440:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
454:class DeploymentConfig(BaseModel):
471:    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
473:    # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound Cloud
Run Job
```

32. user (2026-07-08T05:48:12.810Z)

```
1    """Canonical file-identity hashing for the indexer.
2
3    `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4    (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
`scripts/run_process_and_stitch.py`)
5    defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6    The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7    any future change to one copy would silently diverge the DB key.
8
9    This module is the single source of truth. Import `compute_video_id` everywhere a
10   video needs to be keyed.
11   """
12
13   from __future__ import annotations
14
15   import hashlib
16
17   # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18   # was centralized. The chunk size does not affect the resulting digest ? it only
19   # bounds memory use while streaming large files.
20   _CHUNK_SIZE = 65536
21
22
23   def compute_video_id(filepath: str) -> str:
24       """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26       Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27       into memory at once.
28       """
29       hasher = hashlib.md5()
30       with open(filepath, "rb") as f:
31           for chunk in iter(lambda: f.read(_CHUNK_SIZE), b''):
32               hasher.update(chunk)
```

```
33         return hasher.hexdigest()
34
```

33. user (2026-07-08T05:48:23.759Z)

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker.py. Note: your current working directory is
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer.</tool_use_error>
```

34. user (2026-07-08T05:48:29.298Z)

```
backend\worker\__init__.py
backend\worker\__main__.py
```

35. user (2026-07-08T05:48:36.025Z)

No matches found

36. user (2026-07-08T05:48:41.491Z)

```
Found 5 files
backend\orchestration.py
backend\worker\__main__.py
backend\compute\base.py
backend\compute\cloud_run.py
backend\pipeline\segmenters\trajectory_hybrid.py
```

37. user (2026-07-08T05:48:48.133Z)

```
3:     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
43:from backend.utils.hashing import compute_video_id
101:         # control plane re-hydrating from outputs/<md5>/ restores the video row
126:     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and push
both to
127:     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
147:     uri = f"{out_prefix}/outputs/{video_id}/{fn}"
194:     skip_validation=bool(getattr(args, "skip_validation", False)),
218:     done_uri: str,
226:     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
238:     storage.put(local, done_uri, config=storage_cfg)
239:     logger.info("[worker] wrote completion marker -> %s", done_uri)
241:     logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244:def cmd_infer(args: argparse.Namespace) -> int:
271:     video_id = compute_video_id(local_video)
280:     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
282:     if getattr(args, "skip_validation", False):
284:         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
322:         if getattr(args, "done_uri", None):
324:             args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
394:     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
415:     if getattr(args, "done_uri", None):
417:         args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
609:         video_id = compute_video_id(local_video)
722:         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
727:         help="output prefix (outputs/<video_id>/? is appended)",
754:         "--skip-validation",
```

38. user (2026-07-08T05:48:54.025Z)

```
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
```

```

246     _apply_overrides(config, args.set)
247     if not _run_startup_checks(config):
248         return 2
249
250     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254     compute = get_compute_backend(config)
255     if compute is not None:
256         return _dispatch_remote(compute, args)
257
258     storage_cfg = config.storage.model_dump()
259
260     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261     os.makedirs(scratch, exist_ok=True)
262     config.output.output_dir = scratch
263
264     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265     local_video = storage.fetch(
266         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267     )
268     print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271     video_id = compute_video_id(local_video)
272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275     # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276     # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277     # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278     # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281     # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282     if getattr(args, "skip_validation", False):
283         print(
284             "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285             "gate and already validated this input."
286         )
287         val_results, val_context = [], {}
288     else:
289         val_results, val_context = validator_registry.run_all_with_context(
290             local_video, config
291         )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,

```



```

296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
309         # expires.
310         try:
311             _serialize_and_push_outputs(
312                 scratch,
313                 args.out_uri,
314                 video_id,
315                 [],
316                 "failed",
317                 storage_cfg,
318                 str(getattr(args, "video_uri", "") or ""),
319             )
320             except Exception as e: # never mask the real failure on an artifact-push hiccup
321                 logger.warning("[worker] could not push failure artifacts: %s", e)
322             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
323             if getattr(args, "done_uri", None):
324                 _write_done_marker(
325                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
326                 )
327             return rc
328
329     segments: List[dict] = []
330     segmenter_actual = None
331     segmentation_ran = False
332     status = "failed"
333     try:
334         if failures:
335             print(
336                 "[worker] validation FAILED: "
337                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
338             )
339             return _fail(2)
340         seg_name = args.segmenter or config.indexing.default_segmenter
341         segmenter_cls = segmenter_registry.get(seg_name)
342         if not segmenter_cls:
343             print(
344                 f"[worker] unknown segmenter: {seg_name!r} "

```

39. user (2026-07-08T05:49:05.806Z)

```

6:     --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4 GPU).
7: 2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage layer (D16
? the
73:     argv: List[str],
78:     """Start a Cloud Run Job execution running `python -m backend.worker <argv>`;
return an
84:     times out so the abandoned execution doesn't keep billing the GPU until `--task-

```

```

timeout``.
94:         terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
instead
95:         of bucket-polling the full budget. The done-marker stays the ONLY success signal.
111:         argv: List[str],
137:             run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
235:         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
252:         done_uri = f"{out_root}/_dispatch/{token}.done"
253:         argv: List[str] = [
259:             "--done-uri",
260:             done_uri,
263:             argv += ["--segmenter", spec.segmenter]
267:             argv.append("--skip-validation")
269:             argv += ["--set", kv]
272:             self._job_name, argv, region=self._region, project=self._project
278:             done_uri,
283:             lambda: self._poll_check(done_uri, str(execution or "")),
295:             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
310:             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
320:     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
323:         The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
335:         marker = self._read_marker(done_uri)
345:         marker = self._read_marker(done_uri)
360:         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
376:         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
388:         late = self._read_marker(done_uri)
397:         "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check
"
423:     def _read_marker(self, done_uri: str) -> Optional[dict]:
424:         """"The done-marker dict if the worker has written it, else None ? poll_until keeps
waiting.""""
425:         ref = parse_uri(done_uri)
429:         self._read_json(done_uri) or {}
430:         ) # empty dict still terminates the poll (present == done)

```

40. user (2026-07-08T05:49:06.768Z)

```
backend\utils\paths.py:73:def validate_input_path(
```

41. user (2026-07-08T05:49:13.955Z)

```

225         policy: ExecutionPolicy = DEFAULT_POLICY,
226         token: Optional[str] = None,
227         container_overrides: Optional[tuple[str, ...]] = None,
228     ) -> None:
229         self._client = run_client
230         self._job_name = job_name
231         self._region = region
232         self._project = project
233         self._storage_config = storage_config or {}
234         self._policy = policy
235         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237         self._token = token
238         self._container_overrides = (
239             _DEFAULT_CONTAINER_OVERRIDES
240             if container_overrides is None
241             else tuple(container_overrides)
242         )
243

```

```

244     def run_infer(
245         self,
246         spec: ComputeJobSpec,
247         *,
248         on_wait: "Callable[[float], None] | None" = None,
249     ) -> ComputeResult:
250         out_root = spec.out_uri.rstrip("/")
251         token = self._token or uuid.uuid4().hex
252         done_uri = f"{out_root}/_dispatch/{token}.done"
253         argv: List[str] = [
254             "infer",
255             "--video-uri",
256             spec.video_uri,
257             "--out-uri",
258             spec.out_uri,
259             "--done-uri",
260             done_uri,
261         ]
262         if spec.segmenter:
263             argv += ["--segmenter", spec.segmenter]
264         if spec.skip_validation:
265             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267             argv.append("--skip-validation")
268         for kv in (*spec.config_overrides, *self._container_overrides):
269             argv += ["--set", kv]
270
271         execution = self._client.run_job(
272             self._job_name, argv, region=self._region, project=self._project
273         )
274         LOG.info(
275             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276             self._job_name,
277             execution,
278             done_uri,
279         )
280
281         try:
282             marker = poll_until(
283                 lambda: self._poll_check(done_uri, str(execution or "")),
284                 self._policy,
285                 label="cloud-run-infer",
286                 logger=LOG,
287                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
288                 # job.progress moves during the whole GPU run instead of freezing.
289                 on_tick=on_wait,
290             )
291         except PolicyTimeout as timeout_exc:
292             # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293             # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294             # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296             video_id = str(marker.get("video_id", ""))
297             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299             rc = int(marker.get("returncode", 1))
300             if not video_id:
301                 LOG.warning(

```

```
302         "cloud-run: job=%s completion marker present but empty/unreadable (no "
303         "video_id) ? treating as failure",
304         self._job_name,
305     )
306     rc = rc or 1
307     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309     LOG.info(
310         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311     )
312     return ComputeResult(
313         returncode=rc,
314         outputs_uri=outputs_uri,
315         video_id=video_id,
316         report=report,
317         execution=str(execution or ""),
318     )
319
```

42. user (2026-07-08T05:52:33.527Z)

Structured output provided successfully